Viviane Flor Park
169259

Implementação de Compilador C-

São José dos Campos
2026

Viviane Flor Park
169259

Implementação de Compilador C-

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina Laboratório de Sistemas Computacionais: Compiladores.

Docente: Prof. Dr. Luiz Eduardo Galvão Martins

Universidade Federal de São Paulo

Instituto de Ciência e Tecnologia

São José dos Campos

2026

Sumário

1 INTRODUÇÃO 4

2 PROCESSADOR 5
2.1 Visão Geral 5
2.2 Unidades Funcionais 6
2.2.1 *Program Counter* 6
2.2.2 Memória de Instruções e Unidade de Decodificação 6
2.2.3 Banco de Registradores 7
2.2.4 Unidade Lógica e Aritmética 7
2.2.5 Memória Principal 7
2.2.6 *Current Program Status Register* e Unidade de Validação 7
2.2.7 Unidade de Controle 8
2.2.8 Seletor de *Clock*, Entrada e Saída 8
2.3 Conjunto de Instruções 8
2.3.1 Instruções de Tipo D 9
2.3.2 Instruções de Tipo B 10
2.3.3 Instruções de Tipo M 10
2.3.4 Instruções de Tipo A 11
2.4 Organização da Memória 12

3 COMPILADOR: FASE DE ANÁLISE 13
3.1 Modelagem 13
3.2 Análise Léxica 13
3.3 Análise Sintática 13
3.4 Análise Semântica 13

4 COMPILADOR: FASE DE SÍNTESE 14
4.1 Modelagem 14
4.2 Geração de Código Intermediário 14
4.3 Geração de Código Assembly 14
4.4 Geração de Código Executável 14
4.5 Gerenciamento de Memória 14

5 EXEMPLOS 15
5.1 Código Fonte 15
5.2 Código Intermediário 16

5.3 Código Assembly 16
5.4 Código Executável 16
5.5 16

6 CONCLUSÃO 17

REFERÊNCIAS 18

1 Introdução

2 Processador

2.1 Visão Geral

O processador utilizado para o desenvolvimento deste projeto implementa uma arquitetura ARM monociclo adaptada de 32 bits. Ele conta com 32 registradores de propósito geral, entre os quais o registrador R31 foi reservado para uso como *link register* (LR). Também foram projetados dois registradores de uso dedicado: o *program counter* (PC) e o *current program status register* (CPSR), excepcionalmente de 4 bits. O processador também conta com um sistema de interface com um *field programmable gate array* (FPGA).

O diagrama de blocos do processador, observado na Figura 2, mostra todas as diferentes unidades funcionais que o compõem e suas interações. Detalhes a respeito dos propósitos específicos, funcionamento e interligação dessas unidades são descritos na seção 2.2.

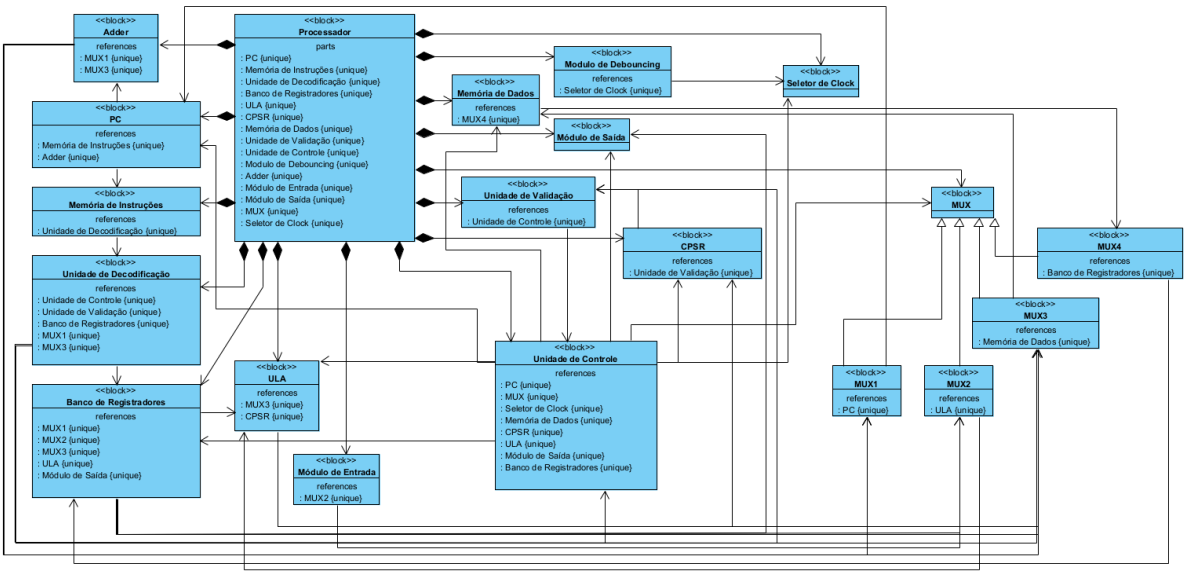


Figura 1 – Diagrama de blocos do processador. Fonte: A autora (2026).

Fazem parte do processador o PC, memória de instruções, unidade de decodificação, banco de registradores, unidade lógica e aritmética (ULA), CPSR, memória de dados, unidade de validação, unidade de controle, multiplexadores e *adder*. Existem quatro instâncias de multiplexadores no projeto, cada qual controla o fluxo de dados em um lugar diferente do *datapath*, observado na Figura 2. Estão presentes no diagrama também os módulos do sistema de interface com FPGA do processador, como os módulos de entrada e saída, seletor de *clock* e unidade de *debouncing*.

2.2 Unidades Funcionais

Para uma visualização mais clara da maneira como as diferentes unidades funcionais do processador se interligam e como ocorre o fluxo de dados, é apresentado na Figura 2 uma visão geral da estrutura do processador.

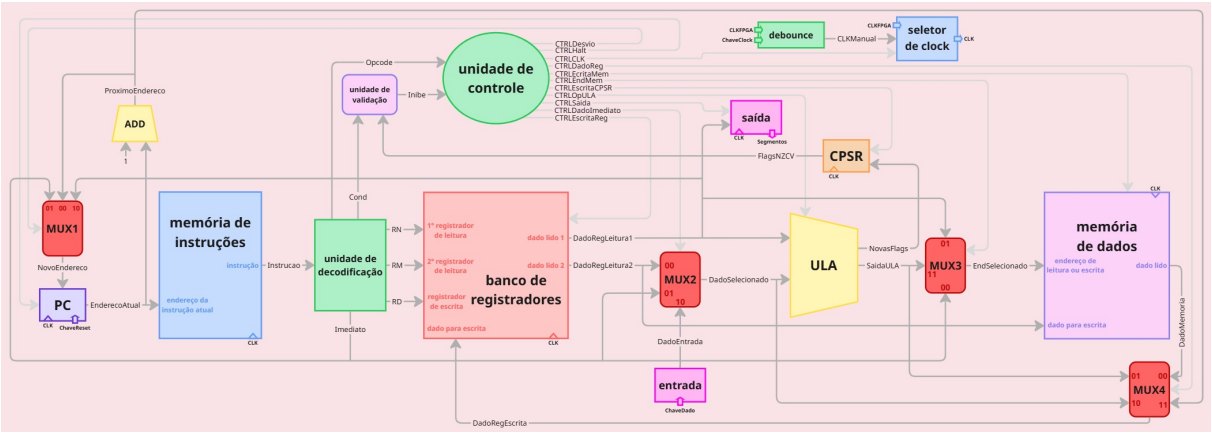


Figura 2 – Visão geral da estrutura processador. Fonte: A autora (2025).

2.2.1 *Program Counter*

O PC é um registrador especial que guarda o endereço da instrução executada. A cada borda de subida do *clock*, essa informação é atualizada seja pelo incremento unitário do *adder*, por um endereço contido em um registrador qualquer do banco de registradores ou por um endereço de desvio contido em uma instrução. Essa decisão é feita por um multiplexador (nesse caso, o MUX1), conforme visto na Figura 2, com o auxílio do sinal CTRLDesvio advindo da unidade de controle.

A atualização desse registrador pode ser inibida pelo sinal de controle CTRLHalt, emitido pela unidade de controle diante de uma instrução HLT, conforme explicitado na seção 2.3. Além disso, seu conteúdo pode ser resetado por meio da entrada ChaveReset, que foi implementada como uma chave de uma placa FPGA.

2.2.2 Memória de Instruções e Unidade de Decodificação

O conteúdo do PC é alimentado à memória de instruções, que lê a instrução contida naquele endereço e a disponibiliza em sua saída. Esse processo é realizada de forma síncrona, com a borda de descida do *clock*.

A instrução, então, é fornecida à unidade de decodificação, que identifica o seu tipo, conforme descrito na seção 2.3, e a particiona nos campos correspondentes.

2.2.3 Banco de Registradores

O banco de registradores recebe dois endereços de leitura e um de escrita, além do dado para a escrita. A leitura dos registradores é feita de forma assíncrona e disponibilizada prontamente nas saídas do banco, conforme visto na Figura 2. A escrita em registradores, por outro lado, é feita de forma síncrona com a borda de subida do *clock* de acordo com o sinal de controle CTRLEscritaReg.

Os dados lidos do banco de registradores são disponibilizados para diversas outras unidades funcionais do processador, como para o próprio PC, conforme já descrito anteriormente, para a ULA, memória de dados ou módulo de saída. Esses dados podem até ser realimentados para o próprio banco.

2.2.4 Unidade Lógica e Aritmética

A ULA tem duas funções principais: realizar operações lógicas e aritméticas e, com base nestas, fazer o cálculo das novas *flags* NZCV do CPSR, conforme descrito na seção 2.2.6. Ela recebe dois operandos, que podem vir do banco de registradores ou da instrução, além do sinal CTRLOpULA, vindo da unidade de controle, que decide qual operação será realizada sobre os dados. A ULA pode realizar 9 operações distintas: soma, subtração (de duas maneiras diferentes, invertendo a ordem dos operandos), multiplicação, divisão, *not*, *and*, *or* e *xor*.

Os resultados da operações realizadas por ela podem ser alimentados à memória principal ou ao banco de registradores. As *flags* calculadas são sempre fornecidas ao CPSR.

2.2.5 Memória Principal

A memória de dados recebe um endereço que pode ser tanto de leitura, quanto de escrita. O conteúdo contido nesse endereço sempre é disponibilizado em sua saída de maneira assíncrona. Já a escrita de um dado neste endereço é feita de forma síncrona com a borda de subida do *clock*, de acordo com o sinal de controle CTRLEscritaMem.

O endereço fornecido à memória pode ter origem na instrução, no caso de endereçamento direto, no banco de registradores, no caso de endereçamento indireto por registrador, ou no resultado de uma operação da ULA, para o endereçamento por deslocamento. A seção 2.3.3 discute mais sobre as diferentes formas de endereçamento à memória às quais o processador oferece suporte.

2.2.6 *Current Program Status Register* e Unidade de Validação

O CPSR e a unidade de validação são, juntas, o coração da execução condicional de instruções, que é o diferencial da arquitetura ARM.

O CPSR é um registrador especial que guarda as *flags* NZCV, quatro bits de informação que dizem respeito ao valor do resultado da ULA. Essa informação consiste nas *flags* N (*negative*), que indica se o resultado é negativo, Z (*zero*), que indica se o resultado é zero, C (*carry*), que indica se o resultado gerou um bit de *carry*, e V (*overflow*), que indica se o resultado estourou seu tamanho máximo.

Conforme visto na Figura 2, as informações do CPSR são alimentadas à unidade de validação junto a um campo especial da instrução, chamado de *cond*. Essas informações são comparadas e, então, é gerado um sinal que decide se a execução da instrução deve ou não ser inibida. A correspondência entre essas informações é discutida com mais detalhes na seção 2.3.

2.2.7 Unidade de Controle

A unidade de controle recebe o *opcode* da instrução e decide, então, a configuração dos sinais de controle. Além disso, caso a unidade de validação tenha gerado um sinal positivo de inibição, ela trata a instrução atual como uma instrução NOP.

2.2.8 Seletor de *Clock*, Entrada e Saída

O seletor de *clock* e os módulos de entrada e saída fazem parte do sistema de interface com FPGA do processador. O primeiro realiza a divisão de frequência do *clock* interno de 50MHz do FPGA, tornando a frequência do *clock* padrão que rege o processamento 1MHz, além de ter o poder de alterar esse *clock* para uma chave da placa, de forma que funcione de maneira manual. Essa segunda funcionalidade do seletor de *clock* é utilizada em instruções IN e OUT.

O módulo de entrada lê um dado codificado em binário nas chaves do FPGA e o disponibiliza para processamento posterior. Ele é utilizado em instruções IN, que guardam essa informação em um registrador. Já o módulo de saída recebe a informação em binário de um registrador e faz sua conversão de modo que possa ser visualizado nos *displays* de 7 segmentos do FPGA.

2.3 Conjunto de Instruções

O conjunto de instruções ao qual o processador oferece suporte pode ser subdividido em quatro categorias: instruções de processamento de dados ou tipo D, de desvio ou tipo B, de acesso à memória ou tipo M e alternativas ou tipo A.

Todas as instruções têm os 6 bits mais significativos em comum. Os quatro primeiros compõem o campo *cond* da instrução, que impõe uma condição de execução sobre ela, e os dois bits seguintes, que compõem o campo op0 e fazem parte do *opcode*, servem para

classificar o tipo da instrução. Essa classificação é importante porque, conforme descrito a seguir, os formatos das instruções variam de acordo com o seu tipo.

2.3.1 Instruções de Tipo D

As instruções de processamento de dados, ou de tipo D, realizam operações aritméticas, lógicas, de movimentação ou de comparação entre valores. Essas instruções estão listadas na Tabela 1.

Vale ressaltar que somente algumas instruções podem alterar as *flags* do CPSR, como as instruções ADDS, SUBS, RSB e CMP. Além disso, todas as instruções de tipo D têm suporte para imediatos no lugar do registrador RM.

Tabela 1 – Tipos, mnemônicos, op1 e descrições das instruções do tipo D.
Fonte: A autora (2025).

Tipo	**Mnemônico**	**op1**	**Descrição**
Aritméticas	ADD	00000	RD <= RN + RM/Im
	ADDS	00001	RD <= RN + RM/Im Altera *flags*
	SUB	00010	RD <= RN - RM/Im
	SUBS	00011	RD <= RN - RM/Im Altera *flags*
	RSB	00100	RD <= RM/Im - RN Altera *flags*
	MUL	00101	RD <= RN · RM/Im
	UDIV	00110	RD <= RN ÷ RM/Im
Lógicas	NOT	00111	RD <= RM/Im
	AND	01000	RD <= RN and RM/Im
	ORR	01001	RD <= RN or RM/Im
	EOR	01010	RD <= RN xor RM/Im
Movimentação	MOV	01011	RN <= RM/Im
Comparação	CMP	01100	RN - RM/Im Altera *flags*

A Tabela 2 mostra o formato das instruções de tipo D. O *opcode* tem tamanho de 8 bits e é dividido em op0, bit I e campo op1. Este último é próprio de cada instrução e é utilizado para identificar a operação realizada. Ele pode ser observado na Tabela 1 junto a cada instrução. Já o bit I é utilizado para identificar se a instrução deve interpretar a informação nos bits 9 a 0 como um registrador ou valor imediato.

Tabela 2 – Formato das instruções do tipo D. Fonte: A autora (2025).

Formato	*cond*	op0 (00)	I	op1	RN	RD	RM/Imediato
Bits	31:28	27:26	25	24:20	19:15	14:10	9:0

2.3.2 Instruções de Tipo B

As instruções de desvio ou *branch*, ou de tipo B, realizam saltos. A lógica da execução condicional de instruções também se aplica a elas, apesar de, a princípio, operarem de forma incondicional. Essas instruções estão listadas na Tabela 3.

Tabela 3 – Mnemônicos e descrições das instruções do tipo B. Fonte: A autora (2025).

Mnemônico	**op1**	**Descrição**
B	00	PC <= Endereço Pula para endereço
BX	01	PC <= RN
BL	10	LR <= PC + 1 PC <= Endereço Pula para endereço

A Tabela 4 mostra o formato das instruções de tipo B. O *opcode* tem tamanho de 4 bits e é dividido nos campos op0 e op1. Cada instrução possui uma configuração única dos bits em op1, conforme visto na Tabela 3, que indica como deve ser executada. Os bits 18 a 0 contêm um endereço absoluto de desvio, e é particularmente relevante nas instruções B e BL.

Tabela 4 – Formato das instruções do tipo B. Fonte: A autora (2025).

Formato	*cond*	op0 (01)	op1	RN	Endereço
Bits	31:28	27:26	25:24	23:19	18:0

2.3.3 Instruções de Tipo M

Instruções de acesso à memória, ou de tipo M, carregam dados da memória principal em registradores e vice-versa. Existem duas instruções de tipo M: LDR (*load*) e STR (*store*). Elas podem ser configuradas de modo que acessem a memória de diferentes maneiras, conforme listado na Tabela 5.

Tabela 5 – Tipos, modos de endereçamento à memória, bits S, I e R e descrições das instruções do tipo M. Fonte: A autora (2025).

Tipo	**Endereçamento**	**bits S, I e R**	**Descrição**
Load	Imediato	010	RD <= **Imediato**
	Direto	000	RD <= Conteúdo do **endereço** de memória contido na instrução
	Indireto	001	RD <= Conteúdo do endereço de memória contido em **RN**
	Deslocamento	011	RD <= Conteúdo do endereço de memória contido em **RN** + *offset* **imediato**
Store	Direto	100	**Endereço** de memória contido na instrução <= RM
	Indireto	101	Endereço de memória contido em **RN** <= RM
	Deslocamento	111	Endereço de memória contido em **RN** + *offset* **imediato** <= RM

A Tabela 6 mostra o formato das instruções de tipo M. O *opcode* tem tamanho de 5 bits e é composto pelo campo op0 e pelos bits S, I e R. Esses bits indicam, repectivamente, se a instrução é de *load* ou *store*, se os bits 12 a 0 são irrelevantes ou se contêm um endereço ou *offset* imediato, e se os bits 22 a 18 contêm um registrador. O bit I é setado para instruções que endereçam a memória de forma direta e por deslocamento, além de ser usado no *load* imediato. O bit R é setado para os endereçamentos indireto e por deslocamento.

Tabela 6 – Formato das instruções do tipo M. Fonte: A autora (2025).

Formato	*cond*	op0 (10)	S	I	R	RN	RD/RM	Imediato/Endereço/ *Offset*
Bits	31:28	27:26	25	24	23	22:18	17:13	12:0

2.3.4 Instruções de Tipo A

As instruções alternativas, ou de tipo A, são aquelas que não se enquadram nas funcionalidades dos outros tipos de instruções descritos. Essas instruções estão listadas na Tabela 7.

Tabela 7 – Mnemônicos, op1 e descrições das instruções do tipo A.
Fonte: A autora (2025).

Mnemônico	**op1**	**Descrição**
NOP	0000	Não faz nada
HLT	0001	Inibe a atualização do PC
IN	0010	Guarda valor de entrada do FPGA em RD
OUT	0011	Faz o display do valor de RN no FPGA

A Tabela 8 mostra o formato das instruções de tipo A. O *opcode* tem tamanho de 6 bits e é composto pelos campos op0 e op1. Cada instrução de tipo A tem um campo op1 único, conforme visto na Tabela 7. Os bits 21 e 17 contêm os registradores RD e RN utilizados nas instruções IN e OUT.

Tabela 8 – Formato das instruções do tipo A. Fonte: A autora (2025).

Formato	*cond*	op0	op1	RN/RD	x
Bits	31:28	27:26	25:22	21:17	16:0

2.4 Organização da Memória

O processador está estruturado de acordo com a arquitetura de Harvard, portanto conta com memórias de instruções e dados separadas, conforme visto nas Figuras 1 e 2. Ambas as memórias armazenam palavras de 32 bits por endereço. Apesar de seus tamanhos poderem ser alterados, para fins desta implementação, a memória de instruções foi configurada com 2^8 endereços, e a memória de programa com 2^{10} endereços.

Como visto na seção 2.3.3, o conjunto de instruções do processador não conta com instruções de manuseio de pilha, por exemplo, o que exige um tratamento especial por parte do programador ou compilador. Igualmente, não há um *stack pointer* (SP), registrador específico para o tratamento de pilha na memória. Conforme já discutido, o único registrador reservado entre os de propósito geral é R31, o *link register*, que tem tratamento específico via *hardware*. Este sempre guarda o valor do *program counter* incrementado de uma unidade diante de uma instrução BL. Vale destacar que, enquanto LR é um ponteiro de memória de instruções, SP seria um ponteiro de memória de dados.

3 Compilador: Fase de Análise

O compilador implementado foi desenvolvido em torno da linguagem de programação C- (LOUDEN, 2004), um subconjunto da linguagem C. Nesta seção, são descritas as convenções léxicas, sintáticas e semânticas que a linguagem define, assim como suas implementações.

3.1 Modelagem

3.2 Análise Léxica

A linguagem C- possui 6 palavras terminais no total: `if`, `else`, `while`, `return`, `void` e `int`. Além disso, ela conta com os símbolos especiais

Tabela 9 – Terminais e símbolos da linguagem C-. Fonte: (LOUDEN, 2004).

Terminais	`if`	`else`	`while`	`return`	`void`	`int`
Símbolos	`+`	`-`				

3.3 Análise Sintática

3.4 Análise Semântica

4 Compilador: Fase de Síntese

Dada a análise feita na primeira fase do compilador, a fase de síntese, detalhada nesta seção, tem como objetivo a geração de código, de fato. Ao final, o propósito do projeto é a geração de linguagem de máquina de acordo com o conjunto de instruções do processador descrito na seção 2.

4.1 Modelagem

4.2 Geração de Código Intermediário

4.3 Geração de Código Assembly

4.4 Geração de Código Executável

4.5 Gerenciamento de Memória

5 Exemplos

5.1 Código Fonte

```
int fact(int n)
{
    if (n <= 0) return 1;
    return n * fact(n - 1);
}

void main(void)
{
    int n;
    n = input();
    output(fact(n));
}
```

```
int gcd(int u, int v)
{
    if (v == 0) return u;
    else return gcd(v, u-u/v*v);
}

void main(void)
{
    int x;
    int y;
    x = input();
    y = input();
    output(gcd(x, y));
}
```

```
int x[10];

int minloc(int a[], int low, int high)
{
    int i; int x; int k;
    k = low;
    x = a[low];
    i = low + 1;
    while(i < high)
    {
        if(a[i] < x)
        {
            x = a[i];
            k = i;
        }
        i = i + 1;
    }
    return k;
}

void sort(int a[], int low, int high)
{
    int i; int k;
```

```
24          i = low;
25          while(i < high-1)
26          {
27              int t;
28              k = minloc(a, i, high);
29              t = a[k];
30              a[k] = a[i];
31              a[i] = t;
32              i = i + 1;
33          }
34      }
35
36      void main(void)
37      {
38          int i;
39          i = 0;
40          while(i < 10)
41          {
42              x[i] = input();
43              i = i + 1;
44          }
45          sort(x, 0, 10);
46          i = 0;
47          while(i < 10)
48          {
49              output(x[i]);
50              i = i + 1;
51          }
52      }
```

5.2 Código Intermediário

5.3 Código Assembly

5.4 Código Executável

5.5 ...

6 Conclusão

Referências

LOUDEN, K. C. *Compiladores: princípios e práticas*. São Paulo: Cengage Learning, 2004. Citado na página 13.